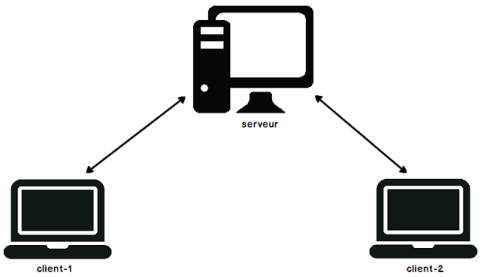
# PROJET RÉSEAUX

## Système de Détection d'Erreurs CRC
## Architecture Client-Routeur-Serveur

Rapport Technique




Réseau client-serveur

**Réalisé par :**

Hamza Elmourabit

Filière: ISIBD S7

**Encadré par :**

Pr.Hnini

**Module :**

Réseaux Informatiques

Année Universitaire : 2025-2026

## Résumé

**Résumé :** Ce projet présente l'implémentation complète d'un système de communication réseau avec détection d'erreurs utilisant le CRC (Cyclic Redundancy Check). L'architecture comprend trois modules : client (génération CRC), routeur (transmission) et serveur (vérification CRC). Implémenté en C avec l'API Winsock, le système démontre pratiquement les principes des communications réseau et de la détection d'erreurs.



**Mots-clés :** CRC, détection d'erreurs, client-serveur, routeur, sockets TCP/IP, Winsock, C, réseau.

# Table des matières


**1 Introduction** **5**
1.1 Contexte et Problématique ..... 5
1.2 Objectifs ..... 5
1.3 Méthodologie ..... 5

**2 Contexte Théorique** **6**
2.1 Les Erreurs de Transmission ..... 6
2.2 Le CRC (Cyclic Redundancy Check) ..... 6
2.2.1 Principe ..... 6
2.2.2 Calcul ..... 6
2.2.3 Polynôme Générateur ..... 6
2.3 Architecture Client-Serveur ..... 7

**3 Architecture du Système** **8**
3.1 Vue d'Ensemble ..... 8
3.2 Composants ..... 8
3.2.1 Client ..... 8
3.2.2 Routeur ..... 8
3.2.3 Serveur ..... 8
3.3 Format des Trames ..... 9
3.3.1 Exemple ..... 9

**4 Implémentation Technique** **10**
4.1 Environnement ..... 10
4.2 Client - Fonction CRC ..... 10
4.3 Serveur - Vérification CRC ..... 11
4.4 Routeur - Multiplexage ..... 11
4.5 Compilation ..... 11
4.6 Exécution ..... 12

**5 Résultats et Tests** **13**
5.1 Protocole de Test ..... 13
5.2 Résultats ..... 13
5.3 Exemples de Sorties ..... 13
5.3.1 Client ..... 13
5.3.2 Routeur ..... 13
5.3.3 Serveur ..... 14

**6 Conclusion et Perspectives** **15**
6.1 Conclusion . . . . . 15
6.2 Perspectives . . . . . 15

**A Code Source Complet** **17**
A.1 Client - client.c . . . . . 17
A.2 Serveur - server.c . . . . . 19
A.3 Routeur - router.c . . . . . 21

# Table des figures

3.1 Architecture générale ..... 8
3.2 Structure d'une trame ..... 9

# Liste des tableaux

4.1 Environnement technique ..... 10

5.1 Résultats des tests ..... 13

# Chapitre 1

# Introduction

## 1.1 Contexte et Problématique

Les communications réseau sont sujettes à diverses perturbations (bruit, interférences) pouvant corrompre les données transmises. Ce projet aborde la problématique de la détection d'erreurs dans les transmissions numériques.

## 1.2 Objectifs

— Implémenter une architecture client-routeur-serveur
— Intégrer un mécanisme CRC de détection d'erreurs
— Développer une solution fonctionnelle en C sous Windows
— Tester et valider le système

## 1.3 Méthodologie

1. Analyse des besoins
2. Conception architecturale
3. Développement des modules
4. Tests et validation
5. Documentation

# Chapitre 2

# Contexte Théorique

## 2.1 Les Erreurs de Transmission

Les erreurs surviennent lorsque les données reçues diffèrent des données envoyées. Causes principales :

- Bruit thermique
- Interférences
- Distorsion
- Évanouissement

## 2.2 Le CRC (Cyclic Redundancy Check)

### 2.2.1 Principe

Code de détection d'erreurs basé sur l'arithmétique polynomiale. Ajoute des bits de redondance (checksum) aux données.

### 2.2.2 Calcul

Pour un message $M(x)$ et polynôme générateur $G(x)$ de degré $n$ :

1. $M(x) \cdot x^n$ (ajouter $n$ zéros)
2. Diviser par $G(x)$
3. Reste $R(x)$ = CRC
4. Transmettre $T(x) = M(x) \cdot x^n + R(x)$

### 2.2.3 Polynôme Générateur

Dans notre projet :

$$G(x) = x^4 + x + 1 \quad \text{(10011 en binaire)}$$

## 2.3 Architecture Client-Serveur

Modèle TCP/IP utilisé :

— Application : Protocole maison (CRC)
— Transport : TCP (communication fiable)
— Réseau : IP (localhost)
— API : Winsock sous Windows

# Chapitre 3

# Architecture du Système

## 3.1 Vue d'Ensemble

Trois processus communiquant via sockets TCP :

FIGURE 3.1 – Architecture générale

## 3.2 Composants

### 3.2.1 Client

— Rôle : Interface utilisateur et génération CRC
— Port : 5000 (vers routeur)
— Fonctions : Saisie, calcul CRC, construction trames

### 3.2.2 Routeur

— Rôle : Intermédiaire
— Ports : 5000 (écoute), 6000 (vers serveur)
— Fonctions : Transmission, multiplexage, corruption optionnelle

### 3.2.3 Serveur

— Rôle : Réception et validation
— Port : 6000 (écoute)
— Fonctions : Vérification CRC, extraction données, réponses

## 3.3 Format des Trames

| 8 bits | 8 bits | 8 bits | N bits | 4 bits | 8 bits |
|---|---|---|---|---|---|
| FLAG | SRC | DST | DATA | CRC | FLAG |

FIGURE 3.2 – Structure d'une trame

### 3.3.1 Exemple

Message "10101010" avec CRC "0011" :

$$00000000S000000SD000000D10101010001100000000$$

# Chapitre 4

# Implémentation Technique

## 4.1 Environnement

| Élément | Version |
|---|---|
| Système | Windows 10/11 |
| Compilateur | MinGW GCC |
| API | Winsock 2.0 |
| Langage | C (C99) |

TABLE 4.1 – Environnement technique

## 4.2 Client - Fonction CRC

```c
char* computeCRC(const char* data) {
    char poly[] = "10011";
    int n = strlen(poly);
    char temp[200] = "";
    strcpy(temp, data);

    for (int i = 0; i < n-1; i++)
        temp[strlen(temp)] = '0';

    for (int i = 0; i < strlen(temp) - n + 1;) {
        if (temp[i] == '1')
            for (int j = 0; j < n; j++)
                temp[i+j] = (temp[i+j]==poly[j])?'0':'1';
        i++;
    }

    char* crc = malloc(5);
    strncpy(crc, temp + strlen(temp) - 4, 4);
    crc[4] = '\0';
    return crc;
}
```

Listing 4.1 – Calcul du CRC

## 4.3 Serveur - Vérification CRC

```
int checkCRC(const char* msgWithCrc) {
    char poly[] = "10011";
    int n = strlen(poly);
    char   temp[200];
    strcpy(temp, msgWithCrc);

    for (int i = 0; i < strlen(temp) - n + 1;) {
        if (temp[i] == '1')
            for (int j = 0; j < n; j++)
                temp[i+j] = (temp[i+j]==poly[j])?'0':'1';
        i++;
    }

    for (int i = strlen(temp)-4; i < strlen(temp); i++)
        if (temp[i] != '0') return 0;
    return 1;
}
```

Listing 4.2 – Vérification CRC

## 4.4 Routeur - Multiplexage

```
while (1) {
    fd_set        readfds;
    FD_ZERO   (&   readfds);
    FD_SET(client_fd, &readfds);
    FD_SET(sock_server, &readfds);

    struct timeval tv;
    tv.tv_sec = 1;
    tv.tv_usec = 0;

    int maxfd = (client_fd > sock_server) ?
                client_fd : sock_server;

    if (select(maxfd+1, &readfds, NULL, NULL, &tv) < 0)
        break;

    if (FD_ISSET(client_fd, &readfds))
        forward_message(client_fd, sock_server);

    if (FD_ISSET(sock_server, &readfds))
        forward_message(sock_server, client_fd);
}
```

Listing 4.3 – Multiplexage avec select()

## 4.5 Compilation

```
@echo off
gcc client.c -o client.exe -lws2_32 -lwsock32
```

```
gcc server.c -o server.exe -lws2_32 -lwsock32
gcc router.c -o router.exe -lws2_32 -lwsock32
echo Compilation reussie!
```

Listing 4.4 – Script de compilation

## 4.6 Exécution

1. Terminal 1 : .\server.exe
2. Terminal 2 : .\router.exe
3. Terminal 3 : .\client.exe

# Chapitre 5

# Résultats et Tests

## 5.1 Protocole de Test

1. Lancement des processus
2. Transmission messages simples
3. Test détection erreurs
4. Mesure performances

## 5.2 Résultats

| Test | Message | CRC | Résultat |
|---|---|---|---|
| T01 | 10101010 | Valide | OK |
| T02 | 11011010 | Valide | OK |
| T03 | [corrompu] | Invalide | **×ERROR** |
| T04 | exit | Valide | →Fin |

TABLE 5.1 – Résultats des tests

## 5.3 Exemples de Sorties

### 5.3.1 Client

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\projet-crc> .\client.exe
Socket created
Connected to server
Enter binary msg: 10101010
```

### 5.3.2 Routeur

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\projet-crc> .\router.exe
Router listening on port 5000 (from client)
Router connected to server on port 6000
Client connected to router
Router forwarding ÔåÆ Server: 00000000S000000SD000000D10101010100100000000
```

### 5.3.3 Serveur

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\projet-crc> .\server.exe
Socket created
Bind success
Server listening on port 6000...
Client connected
```

# Chapitre 6

# Conclusion et Perspectives

## 6.1 Conclusion

Le projet a permis de :

— Implémenter avec succès l'architecture client-routeur-serveur
— Intégrer efficacement le mécanisme CRC
— Démontrer pratiquement la détection d'erreurs
— Produire un système fonctionnel et testé

## 6.2 Perspectives

— Ajouter support multi-clients
— Implémenter CRC-32 plus robuste
— Ajouter correction d'erreurs
— Développer interface graphique
— Portabilité Linux/macOS

# Bibliographie

[1] Tanenbaum, A. S. (2011). *Computer Networks*. Pearson.

[2] Forouzan, B. A. (2012). *Data Communications and Networking*. McGraw-Hill.

[3] Microsoft. (2023). *Winsock Documentation*.

# Annexe A

# Code Source Complet

## A.1 Client - client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <windows.h>

#define MAX 256
#define PORT 5000

#define FLAG "00000000"
#define SRC "S000000S"
#define DST "D000000D"

char* computeCRC(const char* data) {
    char poly[] = "10011";
    int n = strlen(poly);
    char temp[200] = "";
    strcpy(temp, data);
    for (int i = 0; i < n-1; i++) temp[strlen(temp)] = '0';

    for (int i = 0; i < strlen(temp) - n + 1;) {
        if (temp[i] == '1')
            for (int j = 0; j < n; j++)
                temp[i+j] = (temp[i+j] == poly[j]) ? '0' : '1';
        i++;
    }
    char* crc = malloc(5);
    strncpy(crc, temp + strlen(temp) - 4, 4);
    crc[4] = '\0';
    return crc;
}

void func(SOCKET sockfd) {
    char buff[MAX];
    char frame[200];

    for (;;) {
        printf("Enter binary msg: ");
        fgets(buff, MAX, stdin);
```

```
        buff[strcspn(buff, "\n")] = 0;
        if (strlen(buff) == 0) continue;

        strcpy(frame, FLAG SRC DST);
        strcat(frame, buff);
        strcat(frame, computeCRC(buff));
        strcat(frame, FLAG);

        send(sockfd, frame, strlen(frame), 0);

        if (strncmp(buff, "exit", 4) == 0) {
            printf("Client Exit...\n");
            break;
        }

        memset(buff, 0, MAX);
        int bytes = recv(sockfd, buff, MAX-1, 0);
        if (bytes > 0) {
            buff[bytes] = '\0';
            printf("From Server: %s\n", buff);
        }
    }
}

int main() {
    WSADATA wsa;
    SOCKET sockfd;
    struct sockaddr_in servaddr;

    if (WSAStartup(MAKEWORD(2,2), &wsa) != 0) {
        printf("WSAStartup failed: %d\n", WSAGetLastError());
        return 1;
    }

    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd == INVALID_SOCKET) {
        printf("Socket creation failed: %d\n", WSAGetLastError());
        WSACleanup();
        return 1;
    }
    printf("Socket created\n");

    memset(&servaddr, 0, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = inet_addr("127.0.0.1");
    servaddr.sin_port = htons(PORT);

    if (connect(sockfd, (struct sockaddr*)&servaddr, sizeof(servaddr))
        == SOCKET_ERROR) {
        printf("Connection failed: %d\n", WSAGetLastError());
        closesocket(sockfd);
        WSACleanup();
        return 1;
    }
    printf("Connected to server\n");

    func(sockfd);

```

```
    closesocket( sockfd );
    WSACleanup();
    return 0;
}
```

Listing A.1 – Code complet client.c

## A.2 Serveur - server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <windows.h>

#define MAX 256
#define PORT 6000

#define FLAG "00000000"
#define SRC "S000000S"
#define DST "D000000D"

int checkCRC(const char* msgWithCrc) {
    char poly[] = "10011";
    int n = strlen(poly);
    char temp[200];
    strcpy(temp, msgWithCrc);

    for (int i = 0; i < strlen(temp) - n + 1;) {
        if (temp[i] == '1')
            for (int j = 0; j < n; j++)
                temp[i+j] = (temp[i+j] == poly[j]) ? '0' : '1';
        i++;
    }
    for (int i = strlen(temp)-4; i < strlen(temp); i++)
        if (temp[i] != '0') return 0;
    return 1;
}

void func(SOCKET connfd) {
    char buff[MAX];
    int n;

    for (;;) {
        memset(buff, 0, MAX);
        int recvd = recv(connfd, buff, sizeof(buff)-1, 0);
        if (recvd <= 0) break;
        buff[recvd] = '\0';

        int total_len = strlen(buff);
        if (total_len < 24 + 4 + 8) {
            printf("Frame too short!\n");
            continue;
        }

        int payload_len = total_len - 24 - 4 - 8;
```

```
        char msgWithCrc[200] = "";
        strncpy(msgWithCrc, buff + 24, payload_len);
        strncpy(msgWithCrc + payload_len, buff + 24 + payload_len, 4);
        msgWithCrc[payload_len + 4] = '\0';

        printf("Received frame: %s\n", buff);

        if (checkCRC(msgWithCrc)) {
            printf("From client: %.*s\n", payload_len, buff + 24);
        } else {
            printf("CRC ERROR! Message corrupted.\n");
        }

        printf("Enter binary msg: ");
        fgets(buff, MAX, stdin);
        buff[strcspn(buff, "\n")] = 0;

        send(connfd, buff, strlen(buff), 0);

        if (strncmp(buff, "exit", 4) == 0) {
            printf("Server Exit...\n");
            break;
        }
    }
}

int main() {
    WSADATA wsa;
    SOCKET sockfd, connfd;
    struct sockaddr_in servaddr, cli;
    int len = sizeof(cli);

    if (WSAStartup(MAKEWORD(2,2), &wsa) != 0) {
        printf("WSAStartup failed: %d\n", WSAGetLastError());
        return 1;
    }

    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd == INVALID_SOCKET) {
        printf("Socket creation failed: %d\n", WSAGetLastError());
        WSACleanup();
        return 1;
    }
    printf("Socket created\n");

    memset(&servaddr, 0, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = INADDR_ANY;
    servaddr.sin_port = htons(PORT);

    if (bind(sockfd, (struct sockaddr*)&servaddr, sizeof(servaddr)) ==
        SOCKET_ERROR) {
        printf("Bind failed: %d\n", WSAGetLastError());
        closesocket(sockfd);
        WSACleanup();
        return 1;
    }
    printf("Bind success\n");
```

```
    if (listen(sockfd, 5) == SOCKET_ERROR) {
        printf("Listen failed: %d\n", WSAGetLastError());
        closesocket(sockfd);
        WSACleanup();
        return 1;
    }
    printf("Server listening on port %d...\n", PORT);

    connfd = accept(sockfd, (struct sockaddr*)&cli, &len);
    if (connfd == INVALID_SOCKET) {
        printf("Accept failed: %d\n", WSAGetLastError());
        closesocket(sockfd);
        WSACleanup();
        return 1;
    }
    printf("Client connected\n");

    func(connfd);

    closesocket( connfd );
    closesocket( sockfd );
    WSACleanup();
    return 0;
}
```

Listing A.2 – Code complet server.c

## A.3 Routeur - router.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <windows.h>

#define PORT_CLIENT 5000
#define PORT_SERVER 6000
#define MAX 256

int main() {
    WSADATA wsa;
    SOCKET sock_client, sock_server, client_fd;
    struct sockaddr_in addr_client, addr_server, cli;
    int len = sizeof(cli);
    char buff[MAX];
    fd_set readfds;
    struct timeval tv;

    if (WSAStartup(MAKEWORD(2,2), &wsa) != 0) {
        printf("WSAStartup failed: %d\n", WSAGetLastError());
        return 1;
    }

    sock_client = socket(AF_INET, SOCK_STREAM, 0);
    addr_client.sin_family = AF_INET;
```

```
    addr_client.sin_addr.s_addr = INADDR_ANY;
    addr_client.sin_port = htons(PORT_CLIENT);

    bind(sock_client, (struct sockaddr*)&addr_client,
        sizeof(addr_client));
    listen(sock_client, 3);
    printf("Router listening on port %d (from client)\n", PORT_CLIENT);

    sock_server = socket(AF_INET, SOCK_STREAM, 0);
    addr_server.sin_family = AF_INET;
    addr_server.sin_addr.s_addr = inet_addr("127.0.0.1");
    addr_server.sin_port = htons(PORT_SERVER);

    while (connect(sock_server, (struct sockaddr*)&addr_server,
        sizeof(addr_server)) == SOCKET_ERROR) {
        Sleep(100);
    }
    printf("Router connected to server on port %d\n", PORT_SERVER);

    client_fd = accept(sock_client, (struct sockaddr*)&cli, &len);
    printf("Client connected to router\n");

    while (1) {
        FD_ZERO(&readfds);
        FD_SET(client_fd, &readfds);
        FD_SET(sock_server, &readfds);

        tv.tv_sec = 1;
        tv.tv_usec = 0;

        int maxfd = (client_fd > sock_server) ? client_fd : sock_server;

        if (select(maxfd + 1, &readfds, NULL, NULL, &tv) ==
            SOCKET_ERROR) {
            printf("Select error: %d\n", WSAGetLastError());
            break;
        }

        if (FD_ISSET(client_fd, &readfds)) {
            int n = recv(client_fd, buff, MAX-1, 0);
            if (n <= 0) break;
            buff[n] = '\0';

            // Option: corruption pour test
            // if (strlen(buff) > 30) buff[30] =
                (buff[30]=='0')?'1':'0';

            printf("Router forwarding    Server: %s\n", buff);
            send(sock_server, buff, strlen(buff), 0);
        }

        if (FD_ISSET(sock_server, &readfds)) {
            int n = recv(sock_server, buff, MAX-1, 0);
            if (n <= 0) break;
            buff[n] = '\0';
            printf("Router forwarding    Client: %s\n", buff);
            send(client_fd, buff, strlen(buff), 0);
        }
```

```
    }

    closesocket(client_fd);
    closesocket(sock_server);
    closesocket(sock_client);
    WSACleanup();
    printf("Router shutdown\n");
    return 0;
}
```

Listing A.3 – Code complet router.c